

Physics-Informed Neural Networks (PINNs)

Contents

1	Introduction	3
2	PINN vs Traditional Neural Network	3
2.1	Comparison	3
3	Heat Equation Example	4
4	Neural Network Representation	4
4.1	Activation Functions	4
5	PINN Architecture	4
6	Universal Approximation Theorem	5
7	Automatic Differentiation	5
7.1	Advantages	5
8	PDE Residual	5
9	Loss Function	6
9.1	Initial Condition Loss	6
9.2	Boundary Condition Loss	6
9.3	Physics Loss	6
9.4	Total Loss	6
10	PINN Workflow	6
11	Training Algorithm	6
12	Forward and Inverse Problems	7
12.1	Forward Problems	7
12.2	Inverse Problems	7
13	Ball Trajectory Example	8
13.1	Governing Equation	8
13.2	Neural Network Approximation	8
13.3	Physics Residual	8
13.4	Loss Function	8

14 PyTorch Implementation of a PINN	8
14.1 Neural Network Definition	9
14.2 PINN Loss Function	9
14.3 Training Procedure	10
14.4 Explanation	11
15 Advantages of PINNs	11
16 Limitations of PINNs	11
17 Applications of PINNs	12
17.1 Fluid Dynamics	12
17.2 Heat Transfer	12
17.3 Structural Mechanics	12
17.4 Biomedical Engineering	12
17.5 Climate Science	12
18 Conclusion	12

1 Introduction

Physics-Informed Neural Networks (PINNs) are a deep learning framework used to solve differential equations by incorporating physical laws directly into the training process.

Traditional neural networks learn patterns solely from data. In contrast, PINNs combine data with governing physical equations, enabling the network to learn solutions that obey known laws of physics.

A traditional neural network learns a mapping:

$$x \rightarrow y$$

where x is the input and y is the output.

A PINN learns a mapping:

$$(x, t) \rightarrow u(x, t)$$

where:

- x = spatial coordinate
- t = time
- $u(x, t)$ = solution of a differential equation

The major advantage of PINNs is that they can solve problems even when only limited data is available because the governing equations themselves provide supervision.

2 PINN vs Traditional Neural Network

Traditional neural networks rely completely on labeled datasets. During training, they minimize the error between predicted and actual outputs.

$$\text{Loss} = \|y_{\text{pred}} - y_{\text{true}}\|^2$$

This works well when a large amount of data is available.

PINNs extend this concept by introducing physical constraints. Besides fitting data, the network must also satisfy the governing differential equation.

$$\text{Total Loss} = \text{Loss}_{\text{Data}} + \text{Loss}_{\text{Physics}}$$

2.1 Comparison

Traditional Neural Network	Physics-Informed Neural Network
Learns only from data	Learns from data and physics
Requires large datasets	Works with limited data
Ignores governing equations	Enforces governing equations
Data-driven	Physics-driven
Poor extrapolation	Better generalization

3 Heat Equation Example

Consider the one-dimensional heat equation:

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$$

where:

- $u(x, t)$ = temperature
- x = position
- t = time
- α = thermal diffusivity

The objective is to determine the temperature distribution throughout the domain. Instead of numerical methods such as FDM or FEM, a neural network is used:

$$u(x, t) \approx NN(x, t)$$

The network predicts temperature values at any point in space and time.

4 Neural Network Representation

In PINNs, the neural network acts as a continuous function approximator.

$$\text{Input} = (x, t)$$

$$\text{Output} = u_{\theta}(x, t)$$

where θ denotes trainable weights and biases.

The network approximates the unknown solution of the differential equation.

4.1 Activation Functions

Smooth activation functions are preferred:

- Tanh
- Sine

ReLU is generally avoided because higher-order derivatives are not smooth.

5 PINN Architecture

A typical PINN uses a fully connected feed-forward neural network:

$$(x, t) \rightarrow \text{Hidden Layers} \rightarrow u_{\theta}(x, t)$$

Common choices:

- 4–10 hidden layers
- 20–100 neurons per layer
- Tanh activation

6 Universal Approximation Theorem

The theoretical foundation of PINNs comes from the Universal Approximation Theorem.

Statement:

A neural network with at least one hidden layer and a sufficient number of neurons can approximate any continuous function on a bounded domain.

Mathematically,

$$f(x) \approx NN(x)$$

Since PDE solutions are continuous functions, neural networks can theoretically approximate them.

7 Automatic Differentiation

To evaluate differential equations, derivatives of the neural network output are required.

For the heat equation:

$$u_t = \frac{\partial u}{\partial t}$$

$$u_x = \frac{\partial u}{\partial x}$$

$$u_{xx} = \frac{\partial^2 u}{\partial x^2}$$

Frameworks such as PyTorch compute these derivatives automatically through the computational graph.

7.1 Advantages

- High accuracy
- Efficient computation
- Supports higher-order derivatives

8 PDE Residual

The governing equation can be rewritten as a residual.

Starting with:

$$u_t = \alpha u_{xx}$$

Move all terms to one side:

$$f(x, t) = u_t - \alpha u_{xx}$$

If the equation is perfectly satisfied,

$$f(x, t) = 0$$

The residual measures how strongly the network violates the PDE.

9 Loss Function

PINNs usually consist of three loss components.

9.1 Initial Condition Loss

$$L_{IC} = \frac{1}{N} \sum (u_{\text{pred}} - u_{\text{true}})^2$$

9.2 Boundary Condition Loss

$$L_{BC} = \frac{1}{N} \sum (u_{\text{pred}} - u_{\text{true}})^2$$

9.3 Physics Loss

$$L_{PDE} = \frac{1}{N} \sum f(x, t)^2$$

9.4 Total Loss

$$L = L_{IC} + L_{BC} + L_{PDE}$$

The neural network learns by minimizing this combined loss.

10 PINN Workflow

1. Initialize network parameters.
2. Generate training points.
3. Perform forward pass.
4. Compute derivatives using AD.
5. Evaluate PDE residual.
6. Compute total loss.
7. Update parameters using optimizer.
8. Repeat until convergence.

11 Training Algorithm

Step 1: Network Initialization

Randomly initialize weights and biases.

Step 2: Data Generation

Generate:

- Initial condition points
- Boundary condition points
- Collocation points

Step 3: Forward Pass

Predict:

$$u(x, t)$$

Step 4: Automatic Differentiation

Compute:

$$u_t, \quad u_x, \quad u_{xx}$$

Step 5: Residual Calculation

Evaluate:

$$f(x, t)$$

Step 6: Loss Computation

Calculate:

$$L = L_{IC} + L_{BC} + L_{PDE}$$

Step 7: Optimization

Use Adam or L-BFGS.

Step 8: Iteration

Repeat until convergence.

12 Forward and Inverse Problems

12.1 Forward Problems

Known:

$$u_t = \alpha u_{xx}$$

where α is known.

Find:

$$u(x, t)$$

12.2 Inverse Problems

Known PDE:

$$u_t = \alpha u_{xx}$$

Unknown:

$$\alpha$$

PINN simultaneously learns:

$$u(x, t)$$

and

$$\alpha$$

13 Ball Trajectory Example

Consider a ball falling under gravity.

13.1 Governing Equation

$$\frac{d^2y}{dt^2} = -g$$

where:

- $y(t)$ = height
- g = gravity

13.2 Neural Network Approximation

$$y(t) \approx NN(t)$$

13.3 Physics Residual

$$f(t) = \frac{d^2y}{dt^2} + g$$

13.4 Loss Function

$$L = L_{IC} + L_{Physics}$$

where:

$$y(0) = h_0$$

$$\frac{dy}{dt}(0) = v_0$$

The network learns a trajectory that obeys Newton's law.

14 PyTorch Implementation of a PINN

The following implementation demonstrates how a Physics-Informed Neural Network can be used to learn the trajectory of a ball falling under gravity. The network approximates the height function ($y(t)$) while enforcing Newton's law through the physics loss.

14.1 Neural Network Definition

```
import torch
import torch.nn as nn

class BallPINN(nn.Module):
    def __init__(self):
        super().__init__()

        self.net = nn.Sequential(
            nn.Linear(1, 20),
            nn.Tanh(),
            nn.Linear(20, 20),
            nn.Tanh(),
            nn.Linear(20, 1)
        )

    def forward(self, t):
        return self.net(t)
```

14.2 PINN Loss Function

The loss function contains:

- Physics Loss
- Initial Position Loss
- Initial Velocity Loss

```
def pinn_loss(model, t_collocation,
              t0, y0, v0, g=9.81):
    """
    t = t_collocation.clone().requires_grad_(True)

    y = model(t)

    dy_dt = torch.autograd.grad(
        y,
        t,
        grad_outputs=torch.ones_like(y),
        create_graph=True
    )[0]

    d2y_dt2 = torch.autograd.grad(
        dy_dt,
        t,
        grad_outputs=torch.ones_like(dy_dt),
        create_graph=True
    )[0]

    # Physics Loss
    loss_pde = torch.mean(
        (d2y_dt2 + g) ** 2
```

```

)

# Initial Position Loss
y_pred0 = model(t0)

loss_pos = torch.mean(
    (y_pred0 - y0) ** 2
)

# Initial Velocity Loss
dy_pred0 = torch.autograd.grad(
    y_pred0,
    t0,
    grad_outputs=torch.ones_like(y_pred0),
    create_graph=True
)[0]

loss_vel = torch.mean(
    (dy_pred0 - v0) ** 2
)

return (
    loss_pde +
    loss_pos +
    loss_vel
)
'''

```

14.3 Training Procedure

The network is trained using the Adam optimizer. Collocation points are sampled throughout the time domain and used to enforce the governing differential equation.

```

model = BallPINN()

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001
)

# Initial Conditions

t0 = torch.tensor(
    [[0.0]],
    requires_grad=True
)

y0 = torch.tensor([[10.0]])
v0 = torch.tensor([[0.0]])

# Collocation Points

t_collocation = torch.linspace(
    0,
    2,
    100
).view(-1, 1)

```

```

# Training Loop

for epoch in range(2000):

    '''
    optimizer.zero_grad()

    loss = pinn_loss(
        model,
        t_collocation,
        t0,
        y0,
        v0
    )

    loss.backward()

    optimizer.step()

    if epoch % 500 == 0:
        print(
            f"Epoch {epoch}: "
            f"Loss = {loss.item():.6f}"
        )
    '''

```

14.4 Explanation

The network receives time (t) as input and predicts the ball height (y(t)). Automatic differentiation computes the first and second derivatives with respect to time. These derivatives are used to evaluate the residual

$$[f(t) = d^2y_{dt^2+g}]$$

The physics loss forces the network to satisfy Newton's law, while the initial condition losses ensure the correct starting height and velocity. By minimizing the total loss, the PINN learns a physically consistent trajectory for the falling ball.

15 Advantages of PINNs

- Requires very little data.
- Enforces physical laws directly.
- Mesh-free methodology.
- Produces continuous solutions.
- Solves inverse problems.
- Combines machine learning with scientific computing.

16 Limitations of PINNs

- Computationally expensive.
- Slow convergence.

- Difficult for highly complex PDEs.
- Sensitive to architecture design.
- Challenging for high-dimensional problems.

17 Applications of PINNs

17.1 Fluid Dynamics

Solving Navier-Stokes equations.

17.2 Heat Transfer

Temperature distribution modeling.

17.3 Structural Mechanics

Stress and deformation analysis.

17.4 Biomedical Engineering

Blood flow and tissue simulations.

17.5 Climate Science

Weather and atmospheric modeling.

18 Conclusion

Physics-Informed Neural Networks combine deep learning with physical laws. By embedding differential equations directly into the loss function, PINNs can learn physically consistent solutions even with limited data.

The key ideas behind PINNs are:

- Neural network function approximation
- Automatic differentiation
- PDE residual minimization
- Physics-based loss functions

These properties make PINNs a powerful framework for solving forward and inverse scientific problems.